

iOS Production Build & Deployment — Full Guide (React Native + Xcode)

This covers the entire flow: prerequisites → certificates/provisioning → building the release IPA → TestFlight → App Store submission. iOS deployment is more involved than Android because Apple requires signing identities and provisioning profiles tied to your Apple Developer account, managed through Xcode and App Store Connect.

Part 1 — Prerequisites (one-time setup)

Mac + Xcode — You need a Mac (iOS builds cannot be done on Windows/Linux) with Xcode installed from the Mac App Store.

Apple Developer Account — Enroll at developer.apple.com (\$99/year individual or organization account). This gives you access to App Store Connect, certificates, and provisioning profiles.

App Store Connect access — This is the portal (appstoreconnect.apple.com) where you manage app listings, TestFlight, builds, and submissions — separate login/portal from the main Apple Developer site but same Apple ID.

CocoaPods installed — RN iOS projects use CocoaPods for native dependency management:



```
bash
sudo gem install cocoapods
cd ios && pod install
```

Part 2 — Register Your App (App ID / Bundle Identifier)

Go to developer.apple.com → **Certificates, Identifiers & Profiles** → **Identifiers**.

Click + → App IDs → App.

Set a **Bundle ID** (must match `PRODUCT_BUNDLE_IDENTIFIER` in your Xcode project exactly) — e.g., `com.yourcompany.appname`. This should be reverse-DNS style and match what's set in Xcode under your target's **General** → **Identity**.

Enable any **Capabilities** your app needs here (Push Notifications, App Groups, Associated Domains for deep linking, etc.) — these must match what you enable in Xcode later.

Part 3 — Certificates & Signing

There are two main signing approaches:

Option A — Automatic Signing (recommended for most teams)

Open your project in Xcode: `ios/YourApp.xcworkspace` (always open `.xcworkspace`, NOT `.xcodeproj`, once CocoaPods is installed).

Select your target → **Signing & Capabilities** tab.

Check **Automatically manage signing**.

Select your **Team** (your Apple Developer account/organization) from the dropdown.

Xcode will automatically create/manage the Development and Distribution certificates and provisioning profiles for you behind the scenes.

Option B — Manual Signing (needed for CI/CD pipelines, Fastlane, or shared team certs)

In **developer.apple.com** → **Certificates**, create:

An **iOS Distribution Certificate** (for App Store/Ad Hoc release builds).

Download it and double-click to install into your Mac's Keychain (you'll also need the private key — export as a `.p12` if sharing across a team/CI).

Under **Profiles**, create an **App Store Distribution Provisioning Profile**, linking it to your App ID and Distribution Certificate.

Download the `.mobileprovision` file, double-click to install (or reference it manually in Xcode/CI config).

In Xcode, uncheck "Automatically manage signing" and manually select the correct provisioning profile and signing certificate.

(For a mid-level interview, mentioning both — and that CI/CD pipelines typically use manual signing with Fastlane `match` for shared certs across the team — is a strong answer.)

Part 4 — Configure App Version & Build Number

In Xcode, under target → **General**:

Version (`CFBundleShortVersionString`) — user-facing version, e.g. `1.2.0`.

Build (`CFBundleVersion`) — must be incremented for **every** upload to App Store Connect, even for the same version, e.g. `1.2.0 (14)`.

You can also edit these directly in `ios/YourApp/Info.plist`, or via CLI:

```
bash
```

Part 5 — Set Environment / Release Configuration

Ensure any environment-specific config (API base URLs, keys) points to **production** — usually managed via `.env` files with `react-native-config`, or separate Xcode schemes/configurations (Debug vs Release).

Double-check `Info.plist` permissions strings (camera, location, etc.) are filled in with clear descriptions — Apple **rejects** builds with missing usage-description strings for any permission API you use.

Run `pod install` again if you've added any new native dependencies.

Part 6 — Archive the Build

In Xcode's top bar, set the build target device to **"Any iOS Device (arm64)"** — NOT a simulator (you cannot archive for a simulator).

Select **Product → Scheme → Edit Scheme**, ensure the **Archive** action is set to **Release** configuration (not Debug).

Go to **Product → Archive**.

Xcode compiles a release build and, when done, opens the **Organizer** window showing your archive.

(Common gotcha: if "Archive" is greyed out, it's usually because a simulator is selected as the run destination instead of a real device/"Any iOS Device".)

Part 7 — Validate & Upload (Xcode Organizer → App Store Connect)

From the Organizer window, with your archive selected:

Click **Distribute App**.

Choose **App Store Connect → Upload**.

Choose signing option (Automatic recommended if you set up Option A above).

Xcode validates the build (checks entitlements, icons, missing plist strings, etc.) — fix any errors it flags before proceeding.

Click **Upload**. This pushes the build/IPA directly to App Store Connect (no separate manual "export .ipa and upload via Transporter" step needed, though that's an alternative path — see below).

Alternative: Export IPA + Transporter app

Instead of uploading directly from Xcode, you can:

Distribute App → App Store Connect → Export (instead of Upload) to save a `.ipa` file locally. Open the **Transporter** app (free from Mac App Store) → sign in with Apple ID → drag in the `.ipa` → Deliver.

This is useful for CI pipelines that produce an `.ipa` as an artifact and upload it separately.

Part 8 — Processing & TestFlight

After upload, go to **App Store Connect → My Apps → Your App → TestFlight** tab.

The build shows status **"Processing"** for anywhere from a few minutes to ~1 hour.

Once processed, you may need to answer **Export Compliance** questions (encryption usage — most apps just use standard HTTPS, so you typically answer "No" to custom encryption, or declare exempt use).

Add **Internal Testers** (up to 100, must be added as Users with Access in App Store Connect, no review needed, available almost immediately) and/or **External Testers** (up to 10,000, requires a lightweight Beta App Review by Apple, usually approved within 24 hours).

Testers install via the **TestFlight app** using an invite link/code — this is your staging/QA step before public release.

Part 9 — Prepare the App Store Listing

In **App Store Connect → My Apps → Your App → App Store** tab (this is separate from TestFlight):

Fill in **App Information**: name, subtitle, category, content rights.

Add **screenshots** (required for each supported device size — iPhone 6.7", 6.5", iPad if supported, etc.) and optionally an app preview video.

Write the **description, keywords, support URL, marketing URL**.

Set **App Privacy** details (what data you collect — this maps to Apple's privacy nutrition label, required since iOS 14).

Set **Pricing and Availability** (free/paid, which countries).

Under **Build**, select the build you uploaded (the one that finished TestFlight processing).

Part 10 — Submit for Review

Click **Add for Review / Submit for Review**.

Answer **Content Rights** and **Advertising Identifier (IDFA)** usage questions if applicable. Submit.

App goes into **"Waiting for Review" → "In Review"** — Apple's App Review typically takes **24–48 hours** (can be longer, or same-day).

If **rejected**, Apple gives a specific reason under **Resolution Center** — common causes: missing permission-usage descriptions, crashes on review devices, broken links, guideline violations (e.g., referencing beta features, incomplete metadata). Fix and resubmit — you don't need a new build unless the rejection is about the binary itself; metadata-only fixes can often be resubmitted without a new upload.

Once **Approved**, you choose:

Automatic release — goes live immediately upon approval, or

Manual release — you control exactly when it goes live (useful for coordinating with marketing/Android release timing).

Part 11 — Post-Release

Monitor crash reports via **App Store Connect → Analytics** or your crash tool (Crashlytics/Sentry).

For urgent fixes, you must go through this same review process again for a new binary (unlike Android, there's no way to force-push a native update without review) — for JS-only bugs, a CodePush/OTA update (e.g., via Microsoft App Center CodePush or a similar service) can patch JS/asset issues without a new App Store review, but any native code change requires a full new build + review cycle.

Quick Comparison Summary (good for interview soundbite)

Step	Android	iOS
Signing	Keystore (<code>.jks</code>) you generate & keep	Apple-issued Distribution Certificate + Provisioning Profile
Build artifact	<code>.aab</code> (preferred) or <code>.apk</code>	<code>.ipa</code>
Build tool	<code>gradlew bundleRelease</code>	Xcode Archive (or <code>xcodebuild</code> / Fastlane for CI)
Store portal	Google Play Console	App Store Connect
Pre-release testing	Internal/Closed/Open testing tracks	TestFlight (Internal/External)

Step	Android	iOS
Review time	Usually hours	Usually 24–48 hrs, can be longer
Force-update binary without review	Not possible either — but staged rollouts give more control	Not possible for native changes; OTA (CodePush) covers JS-only fixes

Bonus: CLI/Fastlane Mention (nice-to-have for the interview)

For CI/CD pipelines (Bitrise, GitHub Actions, Jenkins), most teams automate this entire flow using **Fastlane**:

`fastlane match` — syncs certificates/profiles across the team via a private git repo, avoiding “it works on my machine” signing issues.

`fastlane gym` — builds and archives the app.

`fastlane pilot` — uploads to TestFlight.

`fastlane deliver` — uploads metadata/screenshots and submits to the App Store.

Mentioning Fastlane, even briefly, signals you understand production-grade release pipelines beyond just clicking through Xcode manually.